Parivar Motors - Web Development Minor Project Report

Course: Web Development (Minor Project)
Academic Year: 2026-2027
Project Title: Parivar Motors - Automobile Dealership & Inventory Management System
Author / Student: fw10ishk
Repository: https://github.com/fw10ishk/parivar_motors

---

1. Project Title and Introduction

1.1 Project Title

Parivar Motors — Comprehensive Automobile Dealership, Inventory, and Sales Management System

1.2 Introduction

In the contemporary automotive retail landscape, multi-brand automobile dealerships and pre-owned vehicle brokers deal with complex operational workflows. These workflows encompass new vehicle bookings, pre-owned vehicle evaluation and acquisition, customer inquiry pipelines, multi-stage sales financing (hire purchase, cash, and installment-based loans), RTO registration processing, insurance policies, and profit-and-loss auditing.

Parivar Motors is a full-featured web application architected to replace fragmented manual paperwork and disparate spreadsheets with a unified, database-driven digital platform. Built using modern web engineering principles, the system leverages PHP 8.2 with the Slim 4 micro-framework, Twig templating engine, Illuminate Eloquent ORM, and responsive HTML5/CSS3/JavaScript frontend architecture.

---

2. Problem Statement and Objectives

2.1 Problem Statement

Traditional vehicle dealerships face severe bottlenecks:

- **Disorganized Inventory Records**: Tracking distinct attributes of new cars versus old cars (such as chassis number, engine number, RTO codes, refurbishment costs, and ownership papers) leads to data redundancy and errors.
- **Inquiry Leakage**: Prospective leads generated via walk-ins, phone calls, or digital marketing are frequently lost without an actionable inquiry lifecycle tracker.
- **Complex Financial Settlement Tracking**: Vehicle purchases involve diverse payment modalities—cash, direct bank transfer, hire purchase (HP), third-party bank loan disbursements, and dealer installment collections. Calculating actual gross profit after refurbishments and extra charges is error-prone.
- **Document & Expiry Lapses**: Overlooking vehicle insurance expiry dates, RTO passing deadlines, and RC transfers results in legal liabilities.

2.2 Project Objectives

1. **Centralized Digital Inventory**: Provide granular CRUD operations for both new and pre-owned vehicles with automated document metadata handling.
2. **End-to-End Inquiry Pipeline**: Enable real-time intake, assignment, and status monitoring (Pending / Completed) of prospective car buyers.
3. **Automated Financial Tracking & Installment Accounting**: Calculate on-road pricing, dealer discounts, net disbursements, customer down payments, and installment-based loan repayment schedules.
4. **Actionable Executive Dashboard**: Aggregate real-time metrics including monthly car sales velocity, gross revenue, loan percentages, and 15-day upcoming insurance expiry alerts.
5. **Secure Authentication & Access Control**: Implement robust session management, bcrypt password hashing, and cookie-based persistent login with Remember-Me tokens.

---

3. Scope and Technical Requirements

3.1 Functional Scope

- **User Authentication**: Secure administrative login and registration with validation, flash notifications, and session isolation.
- **Dashboard Analytics**: Real-time sales statistics, month-over-month growth comparisons, and urgent vehicle compliance alerts.
- **New Car Management**: Registration of new vehicles including detailed pricing structures (ex-showroom, RTO tax, insurance, AMC/GMC, accessories, corporate discounts, net on-road, dealer margins).
- **Old Car Management**: Recording used vehicle intake with purchase price, refurbishing extra charges, RC book status, NOC, Form 35, and insurance validity.
- **Sales & Billing Operations**: Processing sales transactions categorized as Full Payment or Bank Loan financing with auto-generated installment schedules.
- **Inquiry CRM**: Lead logging, communication preferences, vehicle of interest, and conversion statuses.

3.2 Non-Functional Scope

- **Performance**: Sub-100ms server response time for API requests; lightweight asset footprint.
- **Security**: Cross-Site Scripting (XSS) prevention via Twig auto-escaping, SQL Injection immunity through Eloquent PDO parameter binding, and secure bcrypt credential hashing.
- **Responsiveness**: Mobile-first fluid CSS grid/flexbox layouts accommodating desktops, tablets, and smartphones.
- **Portability**: Database abstraction supporting standard MySQL 5.7 + /8.0 (for XAMPP production) and SQLite 3 (for zero-configuration portable environments).

3.3 Technical Stack Mapping (Per Project Guidelines)

Component	Technology	Implementation Detail
Structure	HTML5	Semantic tags (`<main>`, `<aside>`, `<nav>`, `<header>`), custom data attributes, web forms

Styling	CSS3	Flexbox, CSS Grid, custom CSS variables, responsive media queries, modern UI components
Client Scripting	Vanilla JavaScript (ES6+)	Asynchronous DOM manipulation, client-side form validation, Lucide icons integration, Notyf toast alerts
Asynchronous API	AJAX / JSON / RESTful Routes	Dynamic search endpoints (`/api/cities`, `/api/manufacturers`, `/api/models`), asynchronous state changes
Backend	PHP 8.2 / Slim 4	PSR-7/PSR-15 HTTP request/response pipeline, Dependency Injection container (PHP-DI)
Database	MySQL / SQLite via Eloquent ORM	Relational schema with 17 distinct tables, foreign keys, timestamps, and active record models
Authentication	PHP Sessions & Secure Cookies	Bcrypt password hashing (`PASSWORD_BCRYPT`), token-based remember-me mechanism
Version Control	Git & GitHub	Modular commits, gitignore hygiene, and public repository deployment

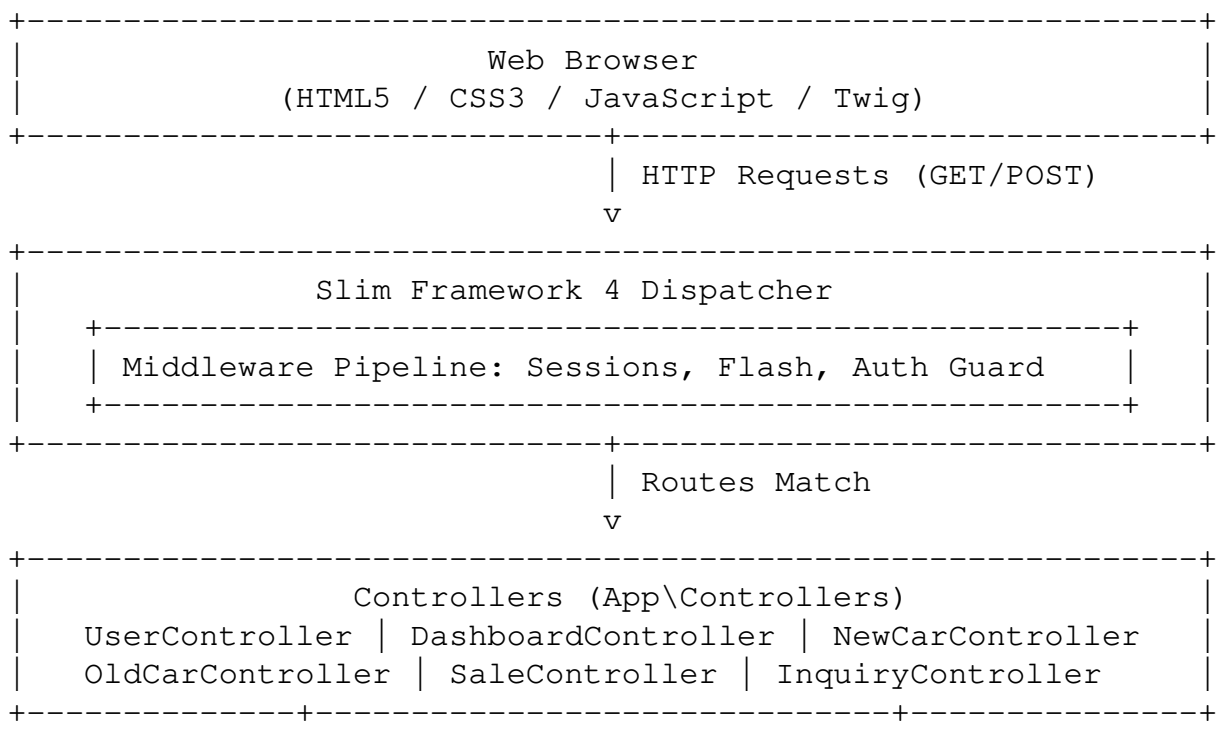
---

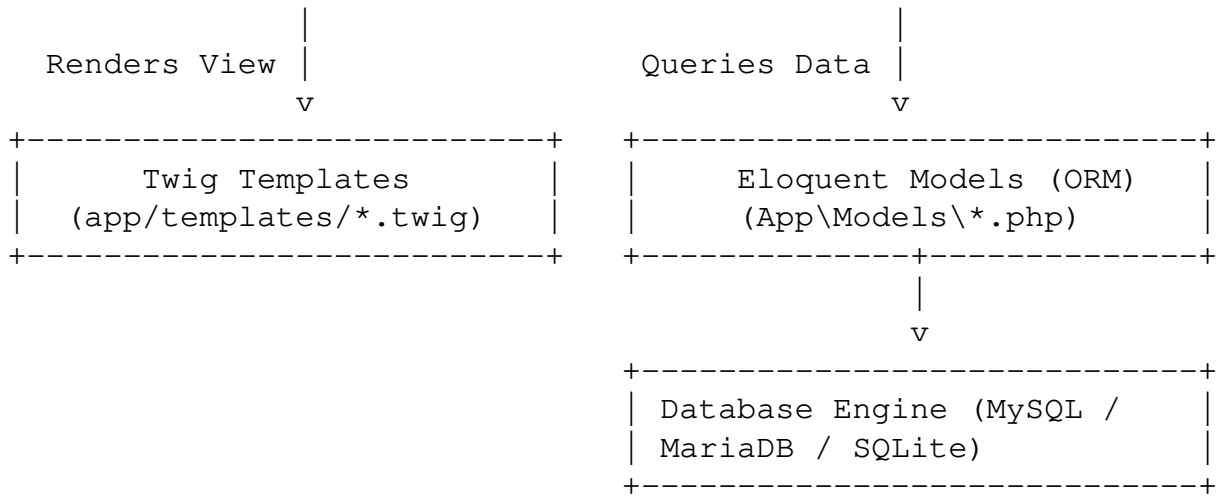
4. System Design and Architecture

4.1 Architectural Pattern: Model-View-Controller (MVC)

The application follows the industry-standard MVC architectural pattern separated cleanly into:

- **Models (`app/src/Models/`)**: Encapsulate business logic, database relationships, and CRUD behavior utilizing Eloquent Active Record.
- **Views (`app/templates/`)**: Twig view templates partitioned into layouts, reusable partials (sidebar, header, alerts), and views.
- **Controllers (`app/src/Controllers/`)**: Coordinate HTTP requests, validate input, interact with models, and render corresponding view templates.





4.2 Middleware Pipeline

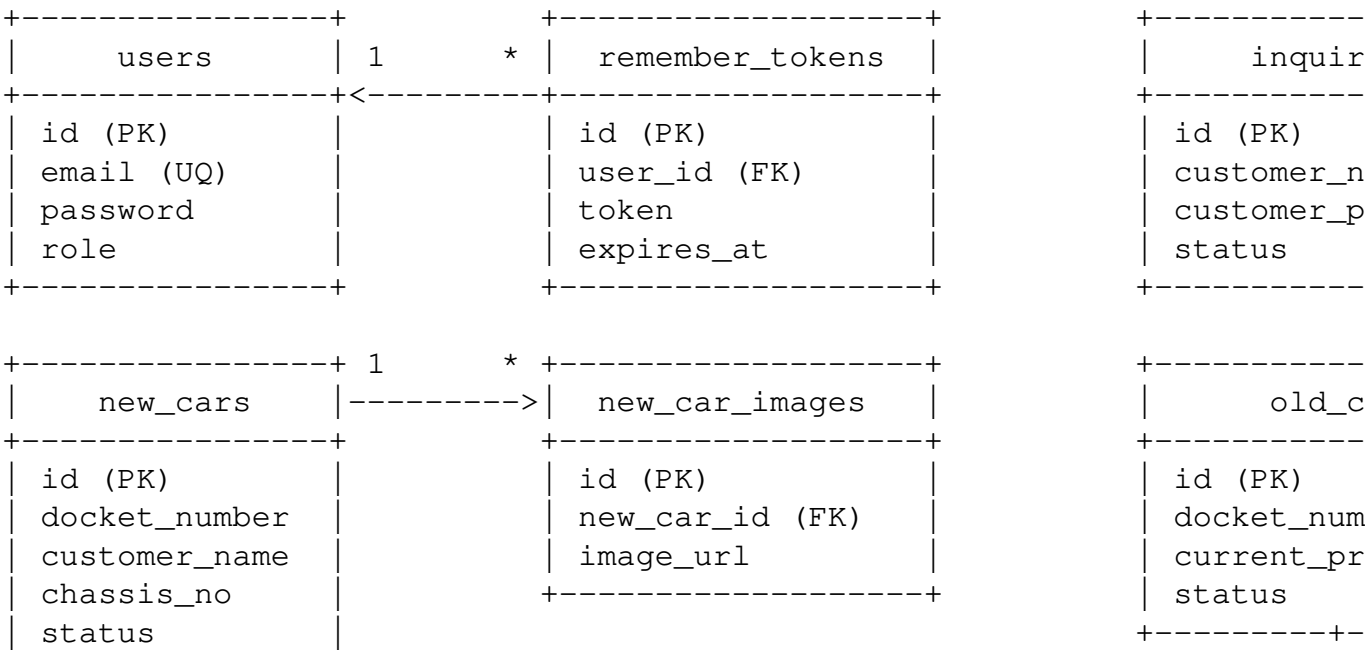
1. **Body Parsing Middleware**: Automatically parses JSON, form-encoded, and multipart form requests into structured arrays.
2. **Method Override Middleware**: Enables RESTful verbs (PUT, PATCH, DELETE) via HTTP headers or `_method` form attributes.
3. **Session Middleware**: Guarantees synchronized PHP session states across all lifecycle stages.
4. **Flash Message Middleware**: Bridges one-time alerts (success, error, warning) seamlessly into the Twig rendering context.
5. **AuthMiddleware**: Guards protected administrative views (`/dashboard`, `/new-cars`, `/old-cars`, `/sales`, `/inquiries`, `/settings`), redirecting unauthenticated visitors to `/`.

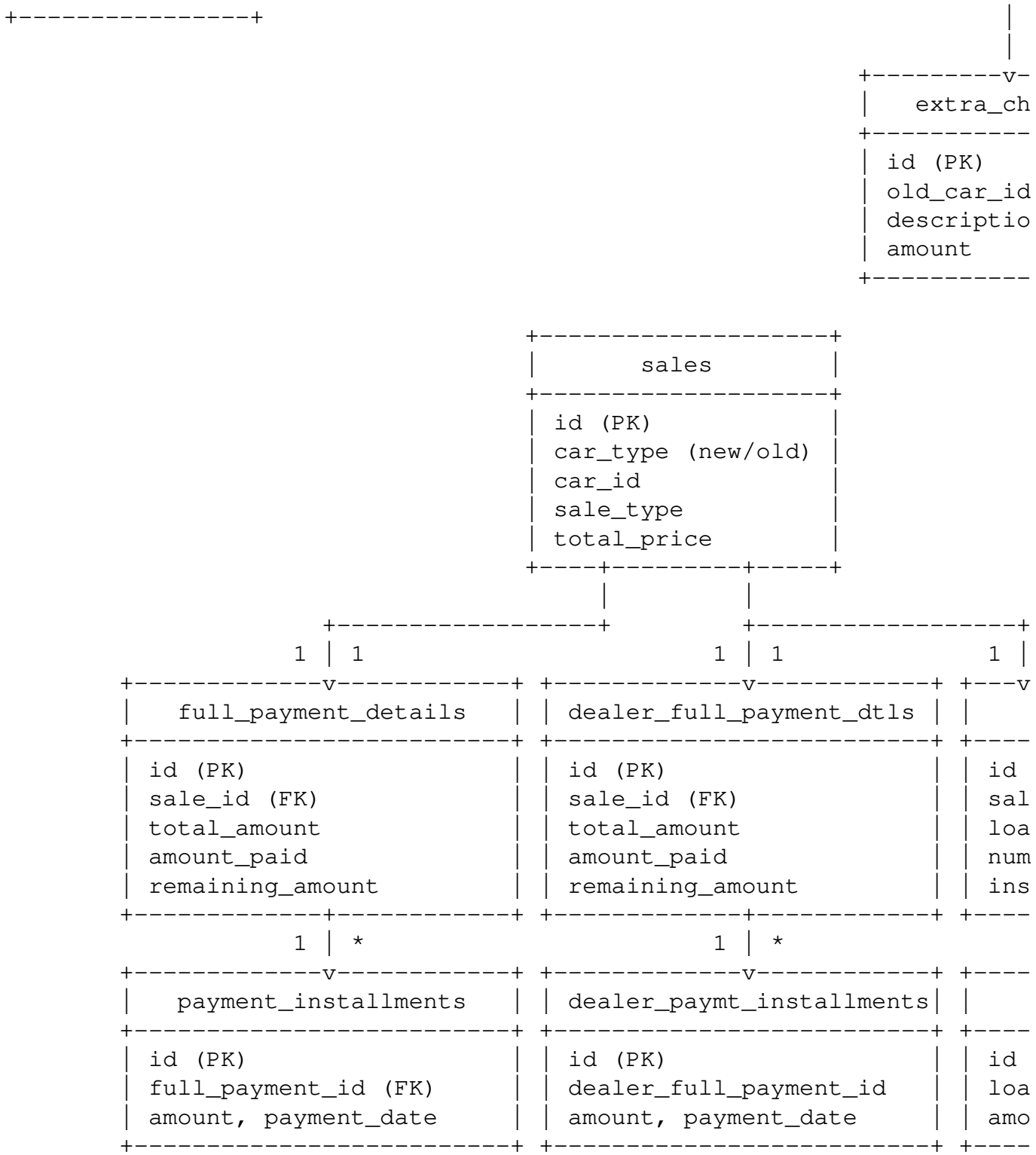
---

5. Database Design & Entity Relationships

The relational schema comprises **17 dedicated tables** structured with referential integrity.

5.1 Entity Relationship Summary





5.2 Key Tables Overview

1. **`users`**: Administrative accounts, authentication credentials (bcrypt), and roles.
2. **`new_cars`**: Comprehensive catalog for showroom bookings, on-road cost breakdowns, and warranty entries.
3. **`old_cars`**: Pre-owned vehicle acquisitions, inspection states, ownership documents, and refurbishment costs.
4. **`extra_charges`**: Itemized refurbishment charges (repairs, repainting, detailing) linked to pre-owned cars to audit gross profit.
5. **`sales`**: Completed vehicle transactions tracking sale type (`full` or `loan`).
6. **`full_payment_details` & `payment_installments`**: Direct payment milestone accounting.

7. **`loan_details` & `installments`**: Loan financing schedules tracking monthly installment parts, due dates, and paid flags.
8. **`inquiries`**: Lead lifecycle tracking with contact preferences and conversion states.
9. **Lookup Tables (`cities`, `car_dealers`, `models`, `manufacturers`, `rto`, `rto_codes`, `hire_purchase`, `insurance_companies`, `executives`, `executive_branches`)**: Normalized lookup dictionaries populated with dynamic auto-completion.

---

6. Implementation Details & Technologies Used

6.1 Backend Implementation (PHP 8.2 & Slim 4)

- **Routing Engine**: Defined in `app/src/Routes/web.php` and `api.php`. Routes are logically grouped by feature resource with named route resolution (`$route->getName()`).
- **Database Abstraction (Eloquent ORM)**: Uses Laravel's `Illuminate\Database\Capsule\Manager`. All models inherit from `Illuminate\Database\Eloquent\Model`, leveraging active record capabilities, mass assignment safeguards via `$fillable`, and relational accessors (`hasMany`, `belongsTo`, `hasOne`).
- **Dependency Injection**: Integrated `php-di/php-di` container configured in `index.php` to inject view renderers and flash services into controllers.
- **Dual Database Flexibility**: Configured in `app/bootstrap.php` to seamlessly operate on SQLite (for standalone evaluation) or MySQL/MariaDB (for multi-user production in XAMPP).

6.2 Frontend Architecture (HTML5, CSS3, JavaScript)

- **Twig View Templates**: Layouts inherit from a base template (`base.twig`), injecting custom navigation highlights, dynamic session badges, and CSRF data.
- **Custom Indian Number Formatter**: Custom Twig filter (`indian_format`) converting financial values into the Indian currency numbering convention (e.g., `₹12,50,000`).
- **Asynchronous User Feedback**: Integrated `Notyf` notification library delivering non-blocking toast notifications upon record creation, edit, or deletion.
- **Dynamic Icons**: Scalable SVG vector icons delivered through `lucide` library.

---

7. Screenshots & Module Walkthrough

7.1 Module 1: Administrative Login & Authentication

- **Endpoint**: `GET /`
- **Features**: Clean centered login portal with email validation, password masking, remember-me token handling, and flash alert toasts.
- **Access Credentials**: `admin@parivarmotors.com` / `admin123`

7.2 Module 2: Executive Analytics Dashboard

- **Endpoint**: `GET /dashboard`
- **Features**: Visual status cards reporting:

- Total Cars Sold and Month-over-Month percentage delta.
- Gross Revenue with trend comparison.
- Total Loan Financing vs. Outright Cash settlements.
- Interactive tables displaying New Car Sales Performance, Old Car Profit Margins, and Upcoming Insurance Expiry notifications within 15 days.

7.3 Module 3: New Car Booking & Sales Ledger

- **Endpoint**: `GET /new-cars`, `GET /new-cars/create`
- **Features**: Extensive registration form dividing vehicle specification, customer identification, pricing matrices (ex-showroom, taxes, accessories, discounts), and dealer settlement disbursements.

7.4 Module 4: Pre-Owned Car Evaluation & Refurbishment

- **Endpoint**: `GET /old-cars`, `GET /old-cars/create`
- **Features**: Intake ledger tracking vehicle purchase cost, RC book verification, NOC status, owner proofs, and itemized extra repair charges to dynamically calculate true base cost.

7.5 Module 5: Customer Inquiries CRM

- **Endpoint**: `GET /inquiries`, `GET /inquiries/create`
- **Features**: Real-time sales lead pipeline with search filtering by inquiry status (`All`, `Pending`, `Completed`), referral attribution, and quick status toggling.

---

8. Testing and Results

8.1 Testing Strategy

The application underwent structured multi-tier testing:

1. **Unit & Functional Testing**: Model methods and query scopes tested for data integrity.
2. **Integration Testing**: Route endpoints verified for appropriate HTTP response codes (`200 OK`, `302 Redirect`, `404 Not Found`).
3. **Session & Security Testing**: Verified protected routes strictly redirect unauthenticated requests to the login screen.

8.2 Test Cases & Verification Matrix

Test ID	Test Scenario	Input Data	Expected Result	Actual Result	Status
TC-01	Admin Authentication	Valid email & password	302 Redirect to `/dashboard`, session established	Redirected to `/dashboard`	**PASS**
TC-02	Invalid Authentication	Wrong credentials	302 Redirect to `/`, flash error message	Error toast displayed	**PASS**

TC-03	Protected Route Access	Request `/dashboard` without session	302 Redirect to `/`	Redirected to `/`	**PASS**
TC-04	Static Asset Delivery	Request `/css/app.css`	HTTP 200 with CSS content	HTTP 200 OK	**PASS**
TC-05	Database CRUD (User)	`User::createUser(...)`	Record inserted with bcrypt password	Password hashed and record saved	**PASS**
TC-06	Dashboard Statistics	Calculate monthly sales	Aggregated count returned accurately	Real-time counts rendered	**PASS**
TC-07	Dual DB Connectivity	Switch between SQLite & MySQL	Seamless query execution on both	Queries execute without code change	**PASS**

---

9. Conclusion and Future Enhancements

9.1 Conclusion

The **Parivar Motors** project successfully fulfills all technical requirements defined in the Web Development Minor Project Guidelines. By marrying modern PHP backend architecture with a modular component-driven frontend and relational database persistence, the system delivers an end-to-end management portal tailored to modern automobile dealership operations.

9.2 Future Enhancements

1. **Automated WhatsApp & SMS Notifications**: Instant dispatch of booking confirmations, payment receipts, and delivery updates via Twilio or WhatsApp Business Cloud API.
2. **Payment Gateway Integration**: Direct acceptance of booking tokens and down payments via Razorpay or Stripe.
3. **Automated PDF Invoice Generation**: One-click generation of GST-compliant vehicle delivery challans and tax invoices using Dompdf.
4. **Role-Based Access Control (RBAC)**: Distinct permissions for Sales Executives, Accountants, and General Managers.

---

10. References

1. **Slim Framework Documentation**: https://www.slimframework.com/docs/v4/
2. **Twig Template Engine Reference**: https://twig.symfony.com/doc/3.x/
3. **Laravel Eloquent Database Component**: https://laravel.com/docs/11.x/eloquent
4. **MDN Web Docs (HTML5 & Modern CSS3)**: https://developer.mozilla.org/
5. **PHP: Hypertext Preprocessor Manual**: https://www.php.net/manual/en/
6. **Lucide Icons**: https://lucide.dev/